

DEEP LEARNING, PYTORCH & APP REVIEW DATA PIPELINE

Introduction to Deep Learning, PyTorch & Google Colab

MDS-1015: Deep Learning and Natural Language Processing

● What Are We Going to Learn?

By the end of this session, students should understand:

- What Deep Learning is
- Why GPUs are useful for Deep Learning
- What Google Colab provides
- What PyTorch is
- What tensors are
- How tensor operations work
- What a computational graph is
- What gradients are
- What automatic differentiation means
- How gradients support neural-network training
- How real-world app reviews become NLP data

KEY IDEA

Data → Tensor → Computation → Gradient → Learning

PART 1

Deep Learning Foundations

● What Is Deep Learning?

Deep Learning is a branch of Machine Learning that uses multi-layer neural networks to learn patterns from data.

Traditional programming

Input + Rules → Output

Machine Learning

Input + Output Examples → Learned Model

Deep Learning

Large Data + Neural Network → Learned Representation + Prediction

Examples

- Image recognition
- Speech recognition
- Machine translation
- Sentiment analysis
- Text generation
- Recommendation systems

● Why Do We Need Neural Networks?

Consider an app-review problem.

We want to determine: Is this review positive or negative?

"The application is excellent and very easy to use."

A traditional rule-based system might look for words such as:

- excellent
- good
- amazing
- bad
- terrible

But language is much more complex. For example:

"The application works, but the delivery experience was terrible."

The system needs to understand relationships between words and context.

Neural networks learn these patterns from examples rather than relying entirely on manually written rules.

● What Makes Deep Learning "Deep"?

A neural network contains layers.



Each layer learns increasingly complex representations. For text:



The word deep refers primarily to the presence of multiple computational layers.

● How Does a Neural Network Learn?

A simplified learning process:

- 1 Give the network an input.
- 2 Network produces a prediction.
- 3 Compare prediction with the expected answer.
- 4 Calculate the error.
- 5 Calculate gradients.
- 6 Update model parameters.
- 7 Repeat.

This is the fundamental training loop behind many Deep Learning systems.

PART 2

Why GPU?

● CPU vs GPU

CPU

- Designed for general-purpose computation
- Strong sequential processing
- Usually has fewer powerful cores

GPU

- Designed for massively parallel computation
- Contains many processing cores
- Very effective for matrix/tensor operations

Deep Learning performs enormous numbers of mathematical operations. Therefore:

Deep Learning workloads can benefit greatly from GPU acceleration.

● Why Matrix Operations Matter

$$z = Wx + b$$

- W = weights
- x = input
- b = bias
- z = resulting value

For thousands or millions of neurons, these become large matrix operations.

GPUs are particularly efficient at performing many such operations in parallel.

● Google Colab

Google Colaboratory (Colab) provides a cloud-based notebook environment.

Students can:

- Write Python code
- Execute code interactively
- Install Python libraries
- Work with datasets
- Use GPUs
- Share notebooks
- Reproduce experiments

The lab specifically asks students to configure the runtime to use a T4 GPU.

● Checking GPU Availability

```
torch.cuda.is_available()
```

"Can PyTorch access a CUDA-enabled GPU?"

If available

- CUDA Available: True

If not

- CUDA Available: False

The notebook's recorded execution shows CPU mode, so students should understand that the result depends on their current Colab runtime configuration.

PART 3

Introduction to PyTorch

● What Is PyTorch?

- Tensor computation
- GPU acceleration
- Automatic differentiation
- Neural-network construction
- Model training
- Research and experimentation

PyTorch is a Deep Learning framework used for the items above.

THE LAB INTRODUCES PYTORCH THROUGH ITS MOST FUNDAMENTAL ABSTRACTION

Tensor

● What Is a Tensor?

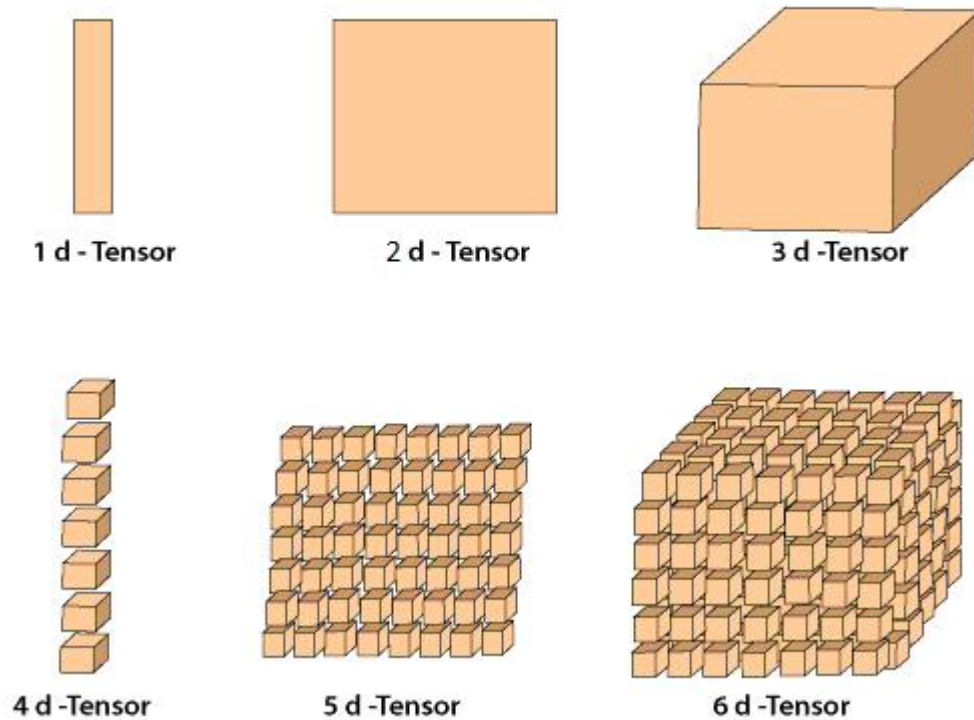
A tensor is a multi-dimensional numerical array. Think of it as a generalization of familiar mathematical structures:

Structure	Example
Scalar	5
Vector	[1, 2, 3]
Matrix	[[1,2],[3,4]]
3D Tensor	Multiple matrices
Higher-dimensional tensor	Multiple dimensions

The notebook describes tensors as N-dimensional arrays designed for GPU computing.

● Tensor

Dimensions of Tensor



Tensor In Tensorflow

Tensors are multi-dimensional arrays used to represent data in TensorFlow

Scaler

1

Vector

$$\begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Matrix

$$\begin{bmatrix} 7 & 2 \\ 8 & 4 \end{bmatrix}$$

Tensor

$$\begin{bmatrix} \begin{bmatrix} 7 & 2 \end{bmatrix} & \begin{bmatrix} 2 & 7 \end{bmatrix} \\ \begin{bmatrix} 8 & 4 \end{bmatrix} & \begin{bmatrix} 4 & 8 \end{bmatrix} \end{bmatrix}$$

● Tensor

1. Tensor = multi-dimensional array of numbers

"grid"

Array Definition

$[S]$

$$\begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

1 2 5 3/4 π

Scalar
(rank 0)

Vector
(rank 1)

$$\begin{bmatrix} m_{11} & m_{12} & \dots & m_{1n} \\ m_{21} & m_{22} & \dots & m_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \dots & m_{nn} \end{bmatrix}$$

$g_{\mu\nu}$

Matrix
(rank 2)

$$T = \begin{bmatrix} X_{111} & X_{112} & X_{113} & \dots & X_{11N} \\ X_{211} & X_{212} & X_{213} & \dots & X_{21N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ X_{N11} & X_{N12} & X_{N13} & \dots & X_{N1N} \end{bmatrix}$$

Rank-3 Tensor
(rank 3)

● Tensor Dimensions

Example	<code>[1, 2, 3]</code>
----------------	------------------------

Shape	<code>(3)</code>
--------------	------------------

Matrix	<code>[1 2 / 3 4]</code>
---------------	--------------------------

Shape	<code>(2, 2)</code>
--------------	---------------------

For a tensor:

Shape = (batch, channels, height, width)

could describe a batch of images.

Important concept: Shape tells us how data is organized.

● Tensor Example from the Lab

The notebook creates:

```
x = torch.tensor([[1.0, 2.0],  
                  [3.0, 4.0]])
```

So:

```
x = [[1, 2], [3, 4]]
```

- Shape: (2, 2)
- Data type: torch.float32

These exact properties are inspected in the notebook.

- # Why Data Type Matters

```
torch.float32
```

A tensor has:

Tensor

values

shape

dtype

device

Understanding these four properties is essential when working with PyTorch.

PART 4

Tensor Operations

● Matrix Multiplication

The notebook performs:

```
z = torch.matmul(x, y)
```

X $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

Y $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Z = XY $\begin{bmatrix} 3 & 3 \\ 7 & 7 \end{bmatrix}$

which matches the notebook output.

- **Why Matrix Multiplication Is Important in Deep Learning**

A neural-network layer can be represented conceptually as:

$$\mathbf{Z} = \mathbf{WX} + \mathbf{b}$$

Therefore, matrix multiplication is not just a mathematical exercise. It is one of the fundamental operations used repeatedly inside neural networks.

Tensor operations → Neural-network computation

● CPU and GPU Tensors

A tensor exists on a particular device. Common devices:

- CPU
- GPU / CUDA

The notebook determines the device:

```
device = torch.device(  
    "cuda" if torch.cuda.is_available() else "cpu"  
)
```

and moves the tensor:

```
x_gpu = x.to(device)
```

● Important Rule: Device Compatibility

Suppose:

- Tensor A $\rightarrow$ GPU
- Tensor B $\rightarrow$ CPU

Trying to perform operations between them can produce a device mismatch error.

Therefore:

Tensors participating in the same computation generally need to be on compatible devices.

This becomes particularly important when training large models.

PART 5

Automatic Differentiation

● What Is a Gradient?

A gradient tells us:

How much does the output change when a parameter changes?

Consider:

$$y = wx + b$$

- $\partial y / \partial w = x$

- $\partial y / \partial b = 1$

For $w = 2$, $x = 3$, $b = 1$, we get: $y = 2(3) + 1 = 7$

● Why Do We Need Gradients?

Suppose a neural network produces the wrong answer. We need to determine:

Which parameters should change, and in which direction?

Gradients provide this information. Very simply:

Gradient → direction and sensitivity of change

This allows optimization algorithms such as gradient descent to update model parameters.

● What Is Autograd?

PyTorch provides automatic differentiation, commonly accessed through autograd.

When PyTorch tracks operations on tensors requiring gradients, it can automatically calculate derivatives.

The lab specifically introduces tensors with:

```
requires_grad=True
```

and then uses:

```
y.backward()
```

to calculate gradients.

- **requires_grad=True**

Consider:

```
w = torch.tensor([2.0], requires_grad=True)
```

"Track operations involving w because I may need its gradient."

Similarly:

```
b = torch.tensor([1.0], requires_grad=True)
```

These represent trainable parameters.

● Forward Pass

The notebook calculates:

$$y = w * x + b$$

This is the forward pass.

Given $w = 2$, $x = 3$, $b = 1$, we calculate: $y = 2(3) + 1 = 7$

Input

$w \times x + b$

Output = 7

● Backward Pass

Then:

```
y.backward()
```

PyTorch calculates derivatives. For $y = wx + b$ we have:

- $\partial y / \partial w = x = 3$
- $\partial y / \partial b = 1$

The notebook produces exactly these gradient values.

● Forward vs Backward

Forward pass

- Input → Prediction

Backward pass

- Prediction → Gradients

Then:

Gradients → Parameter Updates

This gives us the basic conceptual cycle:

Forward → Loss → Backward → Update → Repeat

● From One Equation to Neural Networks

The lab uses:

$$y = wx + b$$

This looks simple, but it is conceptually important. A neural network contains many parameters:

$$w_1, w_2, \dots, w_n$$

Autograd calculates how the output/loss changes with respect to those parameters.

That is why automatic differentiation is fundamental to modern Deep Learning.

PART 6

From Deep Learning to NLP

● What Does This Have to Do with App Reviews?

Our capstone project uses real-world mobile-app reviews.

"The app is very useful and easy to use."

"Worst food service ever."

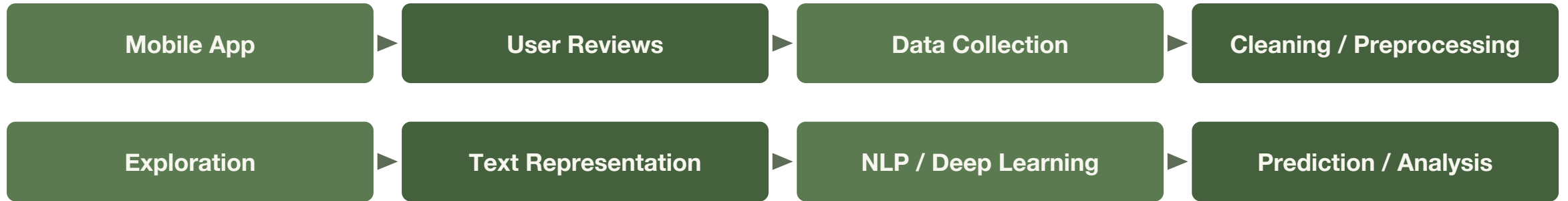
These are examples of natural language data.

The objective is eventually to apply NLP/Deep Learning techniques to such data.

The lab therefore begins by collecting the raw data.

● Data Pipeline

Introduce the overall pipeline:



Today's lab starts near the beginning of this pipeline.

● What Is Data Collection?

Before training a model, we need data. For this project:

App reviews are the raw source of textual data.

Each review may contain:

- User name
- Rating
- Review text
- Timestamp

The notebook demonstrates these fields in a Pandas DataFrame.

● Google Play Scraper

The notebook installs:

```
google-play-scraper
```

and uses the `reviews()` function to retrieve reviews. The selected example is the Pathao application:

```
app_id = 'com.pathao.user'
```

The notebook requests reviews for Bangladesh, sorts them by newest, and initially retrieves five reviews.

● Understanding the DataFrame

The retrieved data is converted into:

```
df = pd.DataFrame(fetched_reviews)
```

Then selected fields are displayed: userName, score, content, at

userName	score	content	at
User A	5	Excellent app...	Date
User B	1	Very bad...	Date
User C	3	Average...	Date

This is the beginning of a real NLP dataset.

PART 7

Ratings as Data

● What Does a Rating Tell Us?

The score field represents the user's rating. Typical values:

1 , 2 , 3 , 4 , 5

A rating provides a useful signal about user satisfaction. For example:

5-star

- strongly positive

1-star

- strongly negative

However, rating and sentiment are not always identical.

● Rating vs Review Text

Consider:

RATING

1

"The application is terrible. It charged me extra."

- Numerical information → rating
- Textual information → review

This makes app reviews useful for combining structured and unstructured data.

● Why Start With 1-Star Reviews?

The lab asks students to:

- 1 Collect 100 reviews.
- 2 Filter reviews with a score of 1.
- 3 Display those reviews.

Why?

Because 1-star reviews provide a relatively clear collection of strongly negative user experiences, which is useful for beginning exploratory NLP analysis.

PART 8

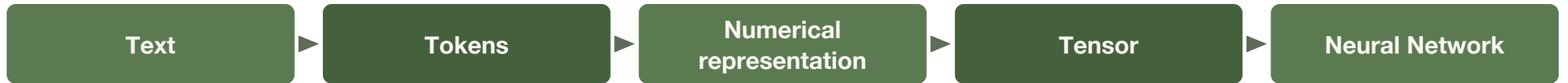
From Raw Reviews to NLP

● Raw Text Is Not Yet a Neural-Network Input

A neural network does not directly understand:

"Worst food service ever."

We eventually need to transform text into numerical representations. Conceptually:



This connects today's two major parts:

PyTorch + App Review Data

● A Simple Example

Review:

| *"The app is excellent"*

Possible conceptual representation:

- "The" → numerical representation
- "app" → numerical representation
- "is" → numerical representation
- "excellent" → numerical representation

These numerical representations can eventually be processed by neural-network architectures.

● Why Tensors and NLP Meet

At the beginning:

Review → Text

After preprocessing and representation:

Review → Numbers → Tensor

Then:

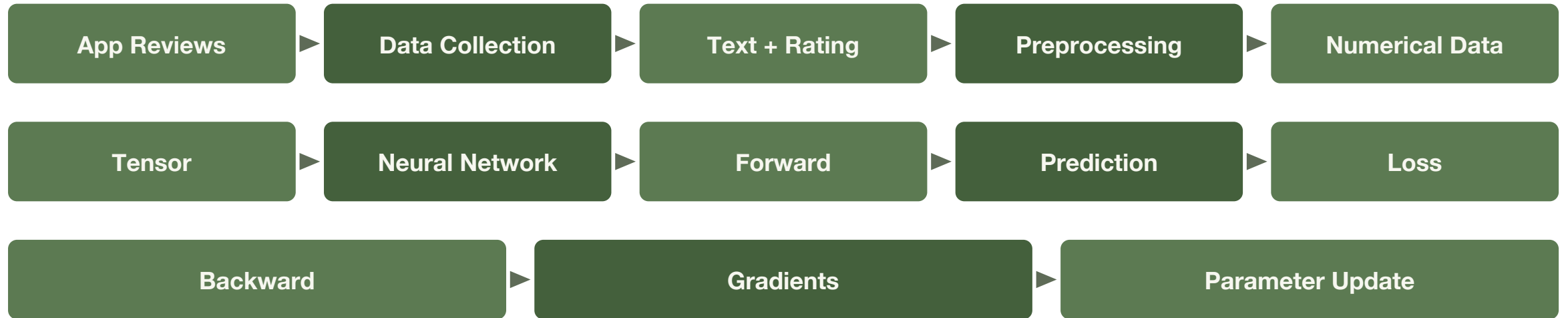
Tensor → Neural Network → Prediction

Therefore, learning tensors today prepares students for later NLP models.

PART 9

Bringing Everything Together

- # The Complete Conceptual Picture



This is the conceptual bridge between the entire lab.

• What We Did in This Lab

1

Environment

Configured Google Colab and checked GPU availability.

2

PyTorch

Created and inspected tensors.

3

Computation

Performed matrix multiplication.

4

Device

Moved tensors to CPU/GPU depending on availability.

5

Autograd

Calculated gradients automatically.

6

Data

Collected real-world app reviews.

7

Project

Started the app-review data pipeline.

All seven activities are directly represented in the uploaded notebook.

● Key Concepts to Remember

Students should be able to explain:

Deep Learning — learning patterns using multi-layer neural networks

GPU — hardware well suited to massively parallel numerical computation

PyTorch — framework for tensor computation and Deep Learning

Tensor — multidimensional numerical data structure

Shape — organization/dimensions of a tensor

Device — where a tensor is stored/executed

Gradient — derivative indicating sensitivity to parameter changes

Autograd — PyTorch's automatic differentiation mechanism

Forward pass — computing an output

Backward pass — computing gradients

App review pipeline — collecting and preparing real-world text data for NLP

● Think About This

Question 1

Why might GPU acceleration become important when we move from a 2×2 tensor to millions of parameters?

Question 2

Why does a neural network need gradients?

Question 3

Why does PyTorch need to know whether a tensor requires gradients?

Question 4

Why can't we directly give raw English sentences to a neural network?

Question 5

What useful information do we obtain from both score and content in an app review?

● Lab Task: Milestone 1

Each group will:

Step 1

Select a mobile application.

Step 2

Find its Google Play package ID.

Step 3

Replace `app_id = 'com.pathao.user'` with the selected application's package ID.

Step 4

Collect 100 reviews.

Step 5

Filter reviews with `score = 1`.

Step 6

Display the selected reviews.

Step 7

Submit the Google Colab notebook link.

These are the exact Milestone 1 requirements in the lab notebook.

Final Takeaway

Today's lab is not just about running PyTorch commands.

It establishes the complete foundation:

Hardware → PyTorch → Tensors → Gradients → Learning

and simultaneously starts the real-world data pipeline:

App → Reviews → DataFrame → NLP Dataset

The important connection for the rest of the course is:

Real-world language data must eventually become numerical representations that Deep Learning models can process.

This would make a strong 45–60 minute theory lecture, followed by the notebook as the hands-on component. A useful next step would be to turn this outline into a fully slide-ready lecture with speaker notes, diagrams, classroom questions, and explanations for every code cell.